

FIRST NAME: Fazel

LAST NAME: Ahmad

SID:

ADVANCED DATA MANAGEMENT 2025/2026
September 9, 2026
CLOSED BOOK

Q1. (up to 10 lines) (only for COMPUTER ENGINEERING STUDENTS)
Explain the quorum principle in leader less replication protocol, providing an example.

Q2. (up to 10 lines)
Explain when indexes should be used in MongoDB and Cassandra.

MongoDB: index attributes used for selection/sorting to avoid expensive collation stores.
Cassandra: first design the primary/partition key according to the workload. Use a secondary index for necessary selection attributes not covered by the primary key, but remember it may still require accessing multiple nodes. Most important difference is MongoDB indexes are commonly used as a general query-performance mechanism. In Cassandra, good primary-key/partition key design comes first and secondary indexes are more limited.

Armed

SID:

only for COMPUTER SCIENCE

containing at least an instance of the relationships are represented in a labelled and show the graphs, clearly explain and

Mobility

mobility: "12"
start Date: "2024-01-1"
end Date: "2030-01-1"
duration: "4 year"

[CAT]

First University

university: "12"
name: "Unige"
country: "Italy"
city: "Genova"

inst

University

en: Mobility

en: Participation

en: Unge

properties on both No

represent the relation in triple

Object

FIRST NAME: Fazel

LAST NAME: Amr

SID:

Q3. (up to 10 lines)

Compare aggregate-oriented and graph data stores with respect to the typically supported data distribution facilities

Aggregate oriented data stores typically provides better support for data distribution because aggregates form natural units for partitioning and can be distributed using partition keys.

Graph data stores are harder to distribute because highly connected nodes may be placed on different machines, resulting in cross-partition relationships and additional network communication during graph traversal. Therefore, aggregate oriented stores are generally better suited for horizontal distribution, whereas graph stores are optimized for traversing highly connected data.

Q4. (up to 10 lines) [only for COMPUTER SCIENCE STUDENTS]

Compare exact k-nearest neighbor (k-NN) search with approximate nearest neighbor (ANN) search. Why do modern vector databases rely on ANN despite its lower accuracy?

Exact k-NN compares the query vector with all stored vectors and returns the true k nearest neighbors providing maximum accuracy but acquiring high computational cost.

ANN search only a subset of promising candidates using specialized indexes, making it much faster and more scalable, although it may miss some true nearest neighbors. Modern vector database rely on ANN because they are more expensive. ANN provides accuracy, latency

nearest Object

Count (DISTINCT ...) as
 sum(home.credits) as tot
 avg(s. convertedCredits) As

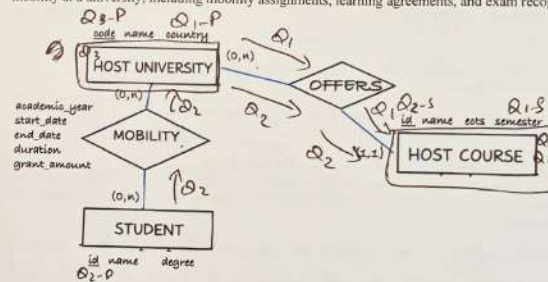
FIRST NAME: Fazel

LAST NAME:

SID:

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following entity relationship diagram, which describes the management of Erasmus student mobility at a university, including mobility assignments, learning agreements, and exam recognition.



The workload is the following:

- Query 1: Determine the countries of the host universities offering at least one course in the second semester.
- Query 2: Given a course, determine all the students that included it in their mobility plan.
- Query 3: Determine the host universities for which at least one mobility has been activated.

Based on the previous ER schema and workload, answer the following questions, motivating all your choices.

Use the aggregate-oriented design methodology (STEP 1-2-3) to design a set of logical aggregates, in meta-notation, for the given domain and the selected workload. Present the output of each step of the methodology.

Course = Host Course

Queries

- Q1 [Course, [Course (semester)]] [Host_University (country) - OFFERS]
 Q2 [Course, [Course (id)]] [Student (id) - OFFERS - MOBILITY]
 Q3 [Host_University, ~~Host_University (code)~~, [], [Host_University (id) - !]

Annotation: please annotate the input ER diagram

Aggregates

Host Course: { id, semester, offers: { country, ~~country~~ } }
 Host_University: { Mobility: { id } } }

Host_University: { code, Mobility: [{ academic_year, start_date }] }

WHERE
RETURN m.mobilityId,
u.name as hostUniversity,
m.duration As duration,
Count(DISTINCT s) as number of Re
sum(home.credits) as total Recognize
avg(s.covetedGrades) As average Co

FIRST NAME: FAZEEN

LAST NAME: AHMED

SID: 5640427

2. Revise and extend the logical aggregate schemas to consider partition keys and super-identifier information.

for HostCourse aggregate:- Partition key = ~~cause-id~~ cause-id
Super identifier = cause-id
This choice efficiently supports Q₂, since the cause is given as input

for HostUniversity aggregate:- Partition key = university-id
Super identifier = university-id
This choice is also used as partition key, so the super identifier coincide
The identifier is also used as partition key, so the super identifier coincide
The query may require access to all nodes and its not efficient -
Q₁ The query is according to semester = 2, while PK is cause-id. Therefore
Q₂ The query is given a specific cause and the value is part of PK.
Q₃ The query is routed to corresponding node and is efficiently supported -
no specific university-id is given. Therefore, all partition nodes must access
no specific university-id is given. Therefore, all partition nodes must access

3. For each aggregate-oriented schema and each associated query, specify the query type (either batch or random-access) and whether it can be efficiently executed on the collection (for random-access queries only).

4. Determine which indexes (if any) should be created to improve the efficiency of workload queries.

Index should be created on HostCourse = semester
This improves Q₁ since semester is used as selection attribute
but is not the Partition Key
No additional indexes are required in Q₂ and Q₃
because causeid is already physical design
to the current partition key
5. Suppose to implement the designed aggregates in MongoDB.
a. Specify which indexes should be mandatorily created and their type (unique/non-unique).
b. Specify Query 2 in CQL and point out how many nodes will be accessed for its execution in MongoDB.

MongoDB automatically creates a unique index on the _id field of each collection. So if we mapped the causeid and University-id then
unique indexes are mapped the causeid and University-id then
(b) Select should, from HostCourse
Where cause id = ?
1 node will be accessed (assuming replication factor is not considered)

To improve Q₁, we can create index on semester
db.HostCourse.createIndex({semester: 1})

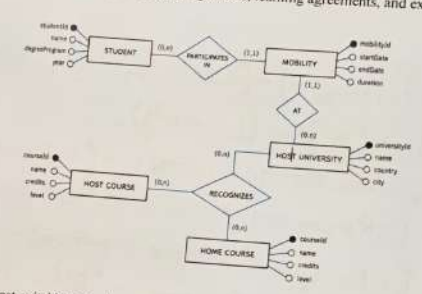
FIRST NAME: Fazel

LAST NAME: Ahmad

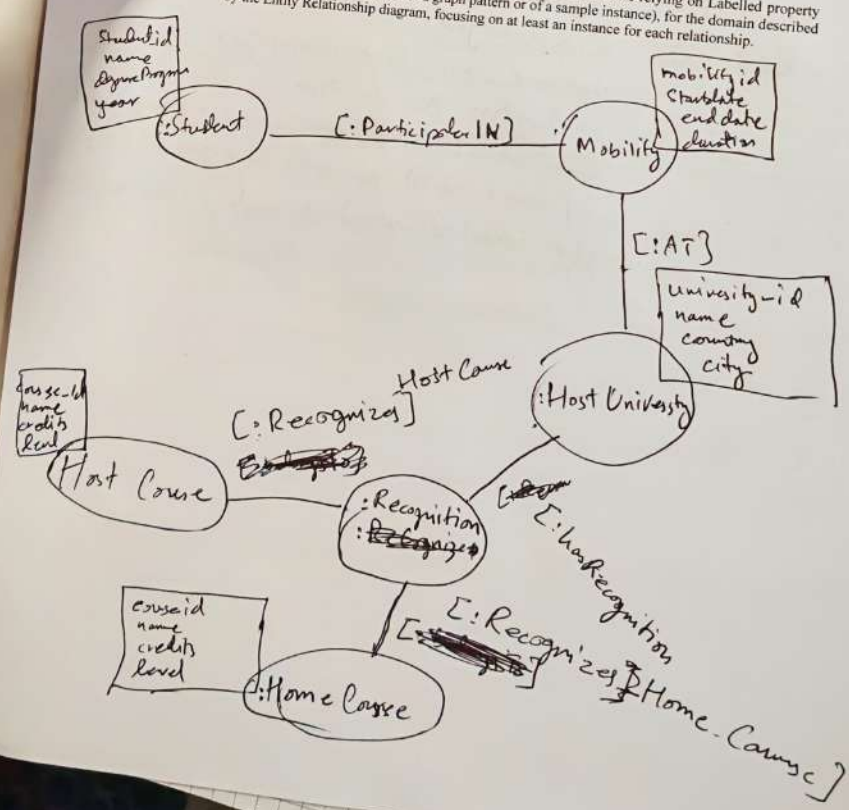
SID:

Exercise 2 [NoSQL graph-based data stores]

Consider the following entity relationship diagram, that describes the management of Erasmus student mobility at a university, including mobility assignments, learning agreements, and exam recognition



Present the most suitable logical schema in a graph-based data store relying on Labelled property graphs, like Neo4j (in the form of a graph pattern or of a sample instance), for the domain described by the Entity Relationship diagram, focusing on at least an instance for each relationship.



b) Provide at least one concrete example of a question or task that requires structured information to support or constrain the retrieval and generation pipeline at a logical LLM-generated response.

to
M
con
in
fint

FIRST NAME: Faisal

LAST NAME: AHMED

SID:

Assuming the information above is represented using a labelled property graph (Neo4j), write a Cypher query that, for each mobility of MSc in Computer Science students, determines the host University, the duration of the mobility, the number of recognized exams, the total recognized cfu, and the average converted grade.

```
MATCH (s:Student)-[:Participate.IN]->(m:Mobility)-
-[:AT]->(u:Host University),
(u)-[:HAS_RECOGNITION]->(r:Recognition),
(r)-[:RECOGNIZES_HOST_COURSE]->(hc:Host Course),
(r)-[:RECOGNIZES_HOME_COURSE]->(hmc:Home Course),
WHERE s.degreeProgram = 'MSc in Computer Science'
RETURN
    m.mobilityId,
    u.name as hostUniversity,
    m.duration as duration,
    Count(DISTINCT r) as number of Recognized Exams,
    sum(hmc.credits) as total Recognized CFU,
    avg(r.convertedGrades) as average Converted Grade.
```

FFERS)
ABILITY)
University (code (id) - !]

id } } }

- year, start lecture }

but remember it may still require
Most important difference is Mongo DB
commonly used as a general query-perf
in Cassandra, good primary-key/partition
first and secondary indexes are m

FIRST NAME: Fazeel LAST NAME: Ahmed SID:

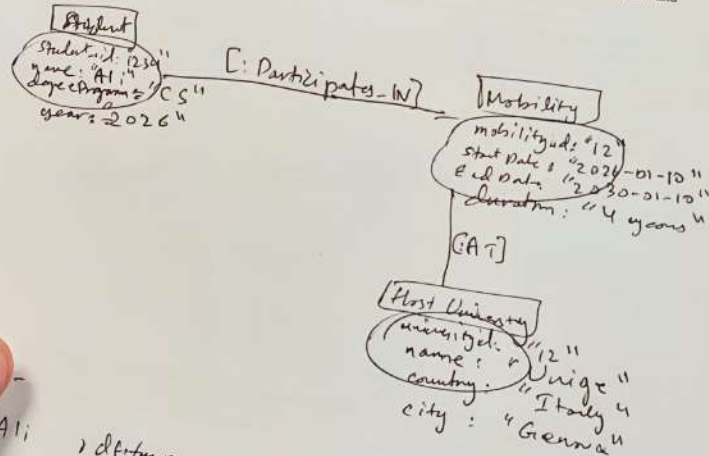
Exercise 2 [NoSQL graph-based data stores]
relationship diagram, that describes the management of Erasmus student
learning agreements, and exam recognition

FIRST NAME: Fazeel LAST NAME: Ahmed SID:

Exercise 3 [Data management for AI driven application] [only for COMPUTER SCIENCE STUDENTS]
With reference to the domain modelled in Exercise 2.1 for LPG:

- Consider one small example instance of your choice (containing at least an instance of the relationships) and discuss the differences in how the relationships are represented in a labelled property graph (e.g., Neo4j) and in RDF. Your answer should show the graphs, clearly explain and compare the modeling choices adopted in each case.

LPG: -



en: AI; rdf:type en: Student
en: Unige rdf:type en: Host University
en: Participation rdf:type en: Mobility
en: AI; rdf:type en: Participation
en: Participation rdf:type AF
en: Unige

In LPG, we can have attributes/properties on both Nodes and edges/relations.
While in RDF, we need to represent the relation in triples in the form of Subject Predicate Object.

...index attributes used for s
to avoid expensive collation stores.
Cassandra: First design the primary/partition key
to be workload Use a secondary index
selection attributes not covered by the
but remember it may still require a sec
Most important difference is Mongo DB index
commonly used as a general query-performance
in Cassandra, good primary-key/partition key de
first and secondary indexes are more

FIRST NAME: Faguel

LAST NAME: Ahmed

Exercise 2 IN-class

FIRST NAME:

LAST NAME:

SID:

udent

2. Assume that the information of the domain modeled in Exercise 2.1 is represented as a graph (e.g., a labelled property graph or an RDF graph). Discuss whether and how such a representation could be exploited in a Retrieval-Augmented Generation (RAG) setting involving Large Language Models (LLMs). In particular:
 - a) Consider the possible role of textual information (e.g., course descriptions, notes, or mobility-related documents) and natural language interaction in this domain. Discuss how an LLM could exploit such information together with the structured information retrieved from the graph, and what advantages this approach might provide compared with directly querying the graph using Cypher or SPARQL.
 - b) Provide at least one concrete example of a question or task for which the graph could be used to retrieve structured information to support or constrain the LLM's output. For your example, sketch the retrieval and generation pipeline at a logical level, from the user's request to the final LLM-generated response.